Langration 4차 예상 질문과 답변

- 질의응답은 10분입니다. 답은 30초 안쪽으로 짧게 하고, 자세한 건 "보고서에 정리해 두었다"로 마무리합니다.
- **답변자**는 역할 기준으로 적었습니다: 백엔드·API → 최우석 / 배포 → 조현주 / 평가·QA → 손채영 / 화면 → 심성욱.
- ⚠ 표시가 있는 답은 수치나 근거를 발표 전에 팀원에게 한 번 확인하세요.

A. 구조 · 백엔드

Q1. 왜 Django를 썼나요? FastAPI가 더 가볍지 않나요?

답변자: 최우석 로그인, 세션, 비밀번호 해시, ORM, 관리자 화면이 Django에 기본으로 들어 있어서 사용자 관리와 대화 저장을 빠르게 만들 수 있었습니다. 저희 서비스는 API 속도보다 답변 생성(LLM)이 대부분의 시간을 차지해서, 프레임워크 속도 차이는 체감되지 않는다고 판단했습니다.

Q2. users와 langration 앱을 왜 나눴나요?

답변자: 최우석 역할이 다르기 때문입니다. users는 회원가입·로그인·로그아웃만, langration은 질문 처리와 대화 세션 관리만 맡습니다. 나눠 두니 로그인 쪽을 고쳐도 질문 처리 코드는 건드리지 않아도 됐고, 작업도 나눠서 할 수 있었습니다.

Q3. 비밀번호는 어떻게 저장하나요?

답변자: 최우석 비밀번호 원문은 저장하지 않고 Django의 해시 함수로 바꿔서 저장합니다. PBKDF2-SHA256 방식이고, 사용자마다 다른 솔트를 붙여 100만 번 반복합니다. 로그인할 때는 입력값을 같은 방식으로 해시해서 비교합니다. 덧붙여, 없는 아이디와 틀린 비밀번호의 응답 시간이 달라서 아이디 존재 여부가 드러나는 문제를 점검에서 찾아, 없는 아이디도 같은 시간이 걸리게 맞췄습니다. 오류 메시지도 둘 다 같게 했습니다.

Q4. 스트리밍은 어떻게 구현했나요? WebSocket이나 SSE를 쓰지 않은 이유는?

답변자: 최우석 Django의 StreamingHttpResponse로 한 줄짜리 JSON(NDJSON)을 단계마다 보냅니다. 그래프는 별도 스레드에서 돌고, 단계가 끝날 때마다 큐에 넣으면 응답이 그걸 꺼내 보냅니다. 오래 걸릴 때 연결이 끊기지 않게 15초마다 빈 줄도 보냅니다. 서버에서 브라우저로 한 방향만 보내면 돼서 WebSocket까지는 필요 없었고, 일반 POST 요청 하나로 끝나서 로그인 세션과 CSRF를 그대로 쓸 수 있었습니다.

Q5. 대화 기억은 어디에 저장되나요?

답변자: 최우석 화면에 보이는 질문과 답 기록은 TiDB의 대화 테이블에 저장돼서, 다시 로그인해도 이전 대화를 불러올 수 있습니다. 그래프가 앞 대화를 기억하는 부분은 LangGraph의 메모리 체크포인터를 쓰고, 사용자와 대화별로 따로 구분합니다.

Q6. Gunicorn worker를 1개로 둔 이유는? 사용자가 많아지면요?

답변자: 최우석 / 조현주 그래프의 대화 기억이 서버 메모리에 있어서, worker가 여러 개면 같은 대화가 다른 worker로 가 기억이 끊길 수 있습니다. 그래서 worker는 1개로 두고, 스레드 4개로 동시 요청을 받게 했습니다. 사용자가 늘면 대화 기억을 메모리가 아니라 DB나 Redis에 두는 체크포인터로 바꾸고, 그다음에 worker와 서버를 늘리는 순서로 확장할 수 있습니다.

Q7. TiDB를 고른 이유는?

답변자: 최우석 MySQL과 호환돼서 Django에서 django-tidb로 바로 쓸 수 있고, 클라우드 무료 구간이 있어 따로 DB 서버를 운영하지 않아도 됐습니다. 팀원 모두 같은 DB에 접속해서 개발할 수 있었던 것도 이유입니다.

B. 배포

Q8. 배포는 어떻게 하나요?

답변자: 조현주 Django(Gunicorn)와 Nginx를 Docker Compose로 묶었습니다. EC2에서 dev 브랜치를 받아 `docker compose up -d --build` 한 번이면 새 버전이 올라갑니다. CSS·JS 같은 정적 파일은 Nginx가 바로 전달하고, 답변 스트리밍은 Nginx가 모아 두지 않도록 버퍼링을 껐습니다.

Q9. HTTPS가 아니던데 보안은 괜찮나요?

답변자: 조현주 지금은 발표용으로 IP 주소에 HTTP로 열어 두었습니다. 실제 서비스라면 도메인을 연결하고 인증서를 붙여 HTTPS로 바꿔야 하고, 그게 다음 단계입니다. 비밀번호는 서버에 해시로만 저장되고, API 키 같은 비밀값은 코드가 아니라 서버 환경 변수로 관리합니다.

Q10. 배포하면서 어려웠던 점은?

답변자: 조현주 / 심성욱 두 가지가 있었습니다. 하나는 Nginx가 응답을 모아서 한 번에 보내서 진행 단계가 실시간으로 안 보인 것인데, 스트리밍 응답에서 버퍼링을 꺼서 해결했습니다. 다른 하나는 새로 배포해도 브라우저에 예전 CSS·JS가 남아 있던 것인데, 강력 새로고침으로 최신 파일을 받게 안내했습니다.

C. 파이프라인 · 분류

Q11. 분류를 왜 코드 규칙으로 먼저 하나요? LLM이 더 똑똑하지 않나요?

답변자: 최우석 / 손채영 코드가 있는지, 옛 import가 있는지는 문자열로 확실히 알 수 있는 정보라, LLM에 맡기면 오히려 가끔 틀렸습니다. 확실한 건 규칙으로 먼저 정하고, 애매한 질문만 LLM이 판단하게 해서 분류가 흔들리지 않게 했습니다. 같은 질문을 3번 보내도 분류가 같은지 보는 항목은 50번 중 50번 같았습니다.

Q12. 코드 없이 "바꿔 줘"라고 하면 어떻게 되나요?

답변자: 최우석 3차에서는 코드가 없어도 변환을 시도해서 엉뚱한 코드를 만들 때가 있었습니다. 4차에서는 분석 단계에서 코드가 없으면 변환하지 않고, 바꿀 코드를 붙여 달라고 안내합니다.

Q13. 근거 데이터는 무엇인가요?

답변자: 손채영 3차에서 만든 데이터를 그대로 씁니다. 옛 import 경로를 1.x 경로로 바꾸는 대응표 9,155건, 메서드 대응표 42건, 최신 공식 문서 271페이지, 마이그레이션 가이드 24페이지입니다. 이걸 Chroma에 넣고 벡터 검색과 키워드(BM25) 검색을 섞어서 찾습니다.

D. 평가

Q14. 같은 모델인데 최종 검증 통과가 53%에서 93%로 오른 이유는? ⚠

답변자: 손채영 모델은 같고, 바뀐 건 모델 바깥의 코드입니다. 질문 분류와 검사·재시도 흐름을 보완한 결과입니다. 같은 30문항을 같은 채점 스크립트로 쟀기 때문에 비교는 공정합니다. ⚠ 어떤 수정이 얼마나 기여했는지는 발표 전에 ③(손채영)이 평가 기록으로 한 번 확인해 주세요. 성적표의 "검증 통과(챗봇 자체 판정) 77%"와 슬라이드의 최종 검증 93%가 다른 기준인지도 함께 확인이 필요합니다.

Q15. 공통 기준은 63%로 아직 낮은데요?

답변자: 손채영 공통 기준은 실행에 필요한 검사에 더해 "정답 import까지 정확히 같아야" 통과라 가장 엄격한 기준입니다. 실행 기준만 보면 93%로, 대부분 실제로 돌아가는 코드입니다. 차이는 정답과 다른, 하지만 동작하는 경로를 고른 경우가 많아서이고, 같은 문제를 일반 LLM에 주면 공통 기준은 0%입니다.

Q16. 일반 LLM보다 답이 느린데요?

답변자: 최우석 네, 중앙값으로 일반 LLM은 3.4초, 저희는 11.5초입니다. 근거를 찾고, 고친 코드를 검사하고, 실패하면 다시 고치는 단계가 있어서 그렇습니다. 대신 일반 LLM은 공통 기준 0%로 옛 코드를 그대로 돌려줄 때가 많았습니다. 기다리는 동안 지금 어떤 단계인지 실시간으로 보여 줘서 멈춘 것처럼 느끼지 않게 했습니다.

Q17. RAGAS 사실 일치가 0.48이면 낮은 것 아닌가요?

답변자: 손채영 사실 일치는 답이 미리 준비한 정답 문장과 얼마나 같은 사실을 담는지 보는 점수라, 표현이 다르거나 정답보다 더 넓게 답하면 점수가 깎입니다. 같은 문항에서 일반 LLM은 0.32였고, 저희가 더 높았습니다. 근거 충실도는 0.89로, 찾은 문서 범위 안에서 답하고 있다는 점이 더 중요한 지표라고 봤습니다.

Q18. 일반 LLM의 근거 충실도가 0인 이유는?

답변자: 손채영 근거 충실도는 "찾은 문서에 있는 내용으로 답했는가"를 봅니다. 일반 LLM은 문서를 찾지 않고 학습한 내용으로만 답하기 때문에, 비교할 근거 문서가 없어 0으로 표시했습니다.

E. 화면 · 기타

Q19. 화면은 프레임워크 없이 만들었나요?

답변자: 심성욱 네, Django 템플릿과 HTML·CSS·JavaScript로 만들었습니다. 화면은 백엔드 API만 불러서 동작해서, 나중에 React 같은 프레임워크로 바꿔도 API는 그대로 쓸 수 있습니다. 답변은 마크다운으로 오는데, 표시할 때 DOMPurify로 위험한 태그를 걸러서 보여 줍니다.

Q20. 3차에서 4차로 오면서 가장 어려웠던 점은?

답변자: 최우석 (필요하면 팀원 보충) Streamlit에서는 화면과 그래프가 한 프로그램이라 신경 쓸 필요가 없던 것들, 예를 들어 로그인 세션, 진행 단계 전달, 대화 저장을 서버와 화면 사이의 API로 직접 설계해야 했던 점입니다. 특히 진행 단계를 실시간으로 보내는 부분은 Django, Nginx, 화면 세 곳을 모두 맞춰야 해서 가장 오래 걸렸습니다.

Q21. 다음에 더 한다면?

답변자: 최우석 세 가지입니다. HTTPS와 도메인 연결, 대화 기억을 DB로 옮겨 서버를 늘릴 수 있게 하는 것, 그리고 버전을 앞뒤로 오가는 변환처럼 아직 약한 유형을 보강하는 것입니다.

답이 막힐 때

- "좋은 질문입니다. 그 부분은 ○○님이 맡았는데, ○○님이 보충해 주시겠어요?"로 담당자에게 넘깁니다.
- 수치가 기억나지 않으면 추측하지 말고 "정확한 수치는 평가 보고서에 있습니다"라고 답합니다.